

LEVEL 4 — DEVELOPMENT TRACK FOUNDATIONS

Databases

Storing, querying and retrieving the data that research and trading systems depend on

Covers: SQL · relational databases (PostgreSQL) · query optimisation · indexing · time-series databases · NoSQL (where appropriate)

Table of Contents

Right-click and choose “Update Field” (or “Update entire table”) after opening in Word to populate page numbers.

Table of Contents	2
12. Databases (Level 4)	3
12.1 Learning Position at a Glance	3
12.2 SQL Fundamentals	3
12.3 Relational Databases (PostgreSQL)	3
12.4 Query Optimisation	4
12.5 Indexing	5
12.6 Time-Series Databases	6
12.7 NoSQL (Where Appropriate)	6
12.8 Worked Exercise — Model Answer	7
12.9 Common Mistakes — Deeper Look	7
12.10 Advanced Extensions	8
12.11 Professional Applications	8

12. Databases (Level 4)

Covers: SQL · relational databases (PostgreSQL) · query optimisation · indexing · time-series databases · NoSQL (where appropriate)

12.1 Learning Position at a Glance

- **What it is:** Systems for storing, querying and retrieving the data that research and trading systems depend on.
- **Why it matters:** Research at scale requires querying years of tick or bar data efficiently; a poorly indexed database silently caps how much research you can do.
- **Prerequisites:** Software Engineering Foundations.
- **Connects to Research:** Well-structured historical data storage is what makes multi-year backtests and cross-sectional research feasible.
- **Connects to Development:** Database schema design and query optimisation are core data-engineering skills.
- **Connects to Trading:** Low-latency access to reference and position data is required for real-time trading systems.
- **Practical exercise:** Design a PostgreSQL schema for storing multi-asset OHLCV data with proper indexing, and write a query that retrieves a cross-sectional universe efficiently.
- **Common beginner mistakes:**
 - Storing time-series data in a schema not indexed by time and instrument, causing slow queries at scale.
 - Reaching for a NoSQL solution without a clear reason when a relational database would suffice.
- **Advanced extensions:** Purpose-built time-series databases (e.g. for tick data) and columnar storage formats.
- **Professional applications:** Historical data storage; feature stores; production reference data.

12.2 SQL Fundamentals

SQL (Structured Query Language) is the standard language for defining, querying and modifying data held in a relational database. A small set of statements covers the vast majority of day-to-day quant research usage: creating tables, inserting or loading data, and, above all, selecting and aggregating data for analysis.

Example 12.1 — A point lookup and a grouped aggregation, the two most common query shapes

```
-- Retrieve the 20 most recent closing prices for a single instrument
SELECT ts, close
FROM ohlcvBars
WHERE instrument_id = 42
ORDER BY ts DESC
LIMIT 20;

-- Aggregate: average daily volume per instrument over the last year
SELECT instrument_id, AVG(volume) AS avg_volume
FROM ohlcvBars
WHERE ts >= NOW() - INTERVAL '1 year'
GROUP BY instrument_id
ORDER BY avg_volume DESC;
```

- **SELECT / WHERE / ORDER BY:** The core of reading data: which columns, which rows, and in what order.
- **GROUP BY / aggregate functions:** Collapsing many rows into a summary per group — AVG, SUM, COUNT, MIN, MAX — the backbone of most research queries.
- **JOIN:** Combining rows from two or more tables based on a shared key, such as linking ohlcvBars to instruments via instrument_id (see §12.3).

12.3 Relational Databases (PostgreSQL)

A relational database organises data into tables of rows and columns, with relationships between tables expressed through keys rather than by duplicating data. PostgreSQL is a mature, open-source relational database widely used in quant research and trading for its combination of reliability, rich SQL support, and strong support for the numeric and time-based data types this domain depends on heavily.

A minimal, correctly-indexed OHLCV schema

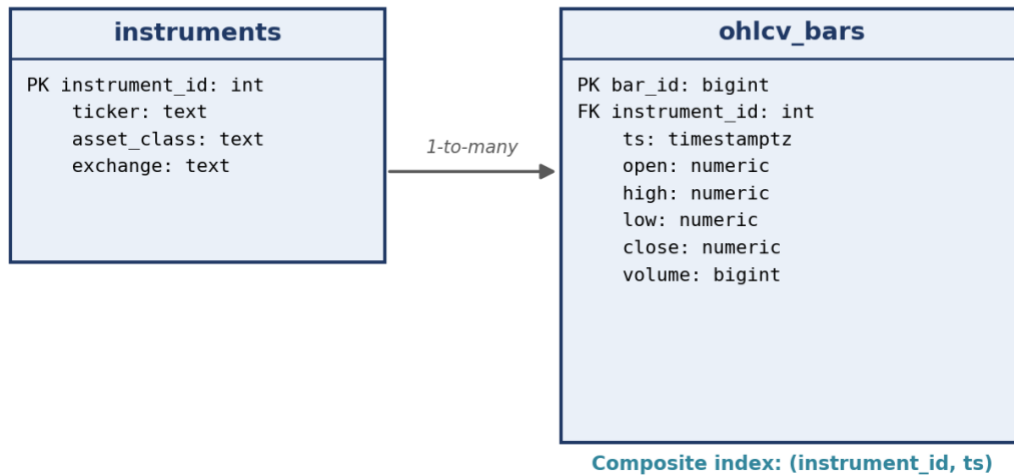


Figure 12.1 — A minimal schema separating slowly-changing reference data (instruments) from high-volume time-series data (ohlcvBars), linked by a foreign key; the composite index on (instrument_id, ts) is what makes both per-instrument and cross-sectional queries fast (§12.5).

Example 12.2 — Creating the two tables from Figure 12.1

```
CREATE TABLE instruments (
    instrument_id SERIAL PRIMARY KEY,
    ticker        TEXT NOT NULL UNIQUE,
    asset_class   TEXT NOT NULL,
    exchange      TEXT NOT NULL
);

CREATE TABLE ohlcvBars (
    bar_id        BIGSERIAL PRIMARY KEY,
    instrument_id INTEGER NOT NULL REFERENCES instruments(instrument_id),
    ts            TIMESTAMPTZ NOT NULL,
    open          NUMERIC NOT NULL,
    high          NUMERIC NOT NULL,
    low           NUMERIC NOT NULL,
    close         NUMERIC NOT NULL,
    volume        BIGINT NOT NULL,
    UNIQUE (instrument_id, ts)
);
```

Separating instruments from ohlcvBars — rather than repeating the ticker and exchange on every single bar row — is the relational-database habit of normalisation: each fact is stored once, so a ticker rename or exchange correction happens in exactly one place rather than requiring an update across millions of historical rows.

12.4 Query Optimisation

A database's query planner chooses how to execute a given SQL query — which indexes to use, which order to join tables in, whether to scan a whole table or use an index. Understanding, at a basic level, how the planner reasons about a query is what separates a query that runs in milliseconds from one that silently takes minutes as a table grows.

Example 12.3 — Using EXPLAIN ANALYZE to see exactly how a query is executed

```
EXPLAIN ANALYZE
SELECT ts, close FROM ohlcvBars
WHERE instrument_id = 42 AND ts >= '2024-01-01';

-- Output (abbreviated):
-- Index Scan using ohlcvBars_instrument_id_ts_idx
--   Index Cond: (instrument_id = 42 AND ts >= '2024-01-01')
--   Planning Time: 0.14 ms
--   Execution Time: 2.31 ms
```

- **Read the query plan, don't guess:** EXPLAIN ANALYZE shows exactly which indexes (if any) were used and how long each step took, replacing intuition with a direct, measured answer.
- **Filter before joining where possible:** Applying a selective WHERE condition as early as possible reduces the number of rows the rest of the query has to process.
- **Avoid functions on indexed columns in WHERE:** A condition like WHERE DATE(ts) = '2024-01-01' prevents the planner from using a plain index on ts; rewriting as a range (ts >= '2024-01-01' AND ts < '2024-01-02') allows the index to be used directly.

12.5 Indexing

An index is a separate, ordered data structure (typically a B-tree) that lets the database locate rows matching a condition without scanning every row in the table — conceptually similar to a book's index letting a reader jump directly to a page rather than reading the whole book to find a topic. Without an appropriate index, every query becomes a full table scan, and query time grows linearly with the size of the table.

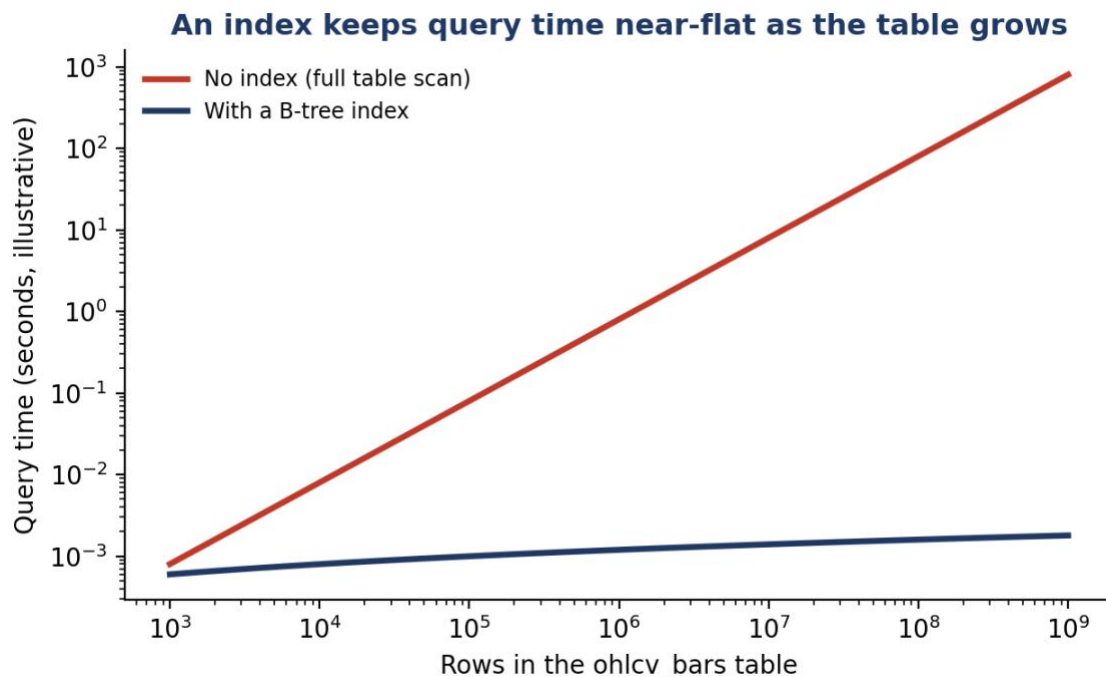


Figure 12.2 — Without an index, query time grows linearly with table size; with a B-tree index, query time grows only logarithmically, staying practically fast even as a table reaches billions of rows.

Example 12.4 — A composite index and why column order matters

```
-- A composite index matching the most common query pattern:
-- "give me this instrument's bars in this time range"
CREATE INDEX ohlcvBars_instrument_ts_idx
ON ohlcvBars (instrument_id, ts);
```

```
-- Column order matters: this index efficiently supports queries
-- filtering on instrument_id alone, or on instrument_id AND ts,
-- but NOT a query that filters on ts alone without instrument_id.
```

- **Index the columns you actually filter and sort by:** An index only helps queries that filter or order by (a prefix of) its columns; an unused index adds write overhead for no benefit.
- **Composite indexes for the common query shape:** For time-series data queried per instrument, an index on (instrument_id, ts) — in that order — directly matches the practical exercise's most common access pattern.
- **Indexes are not free:** Every index must be updated on every insert or update, so indexing every column indiscriminately trades write performance for read performance that may not be needed.

12.6 Time-Series Databases

A time-series database is purpose-built for data that arrives continuously, ordered by time, and is queried predominantly by time range — exactly the shape of tick data. Compared to a general-purpose relational database, time-series databases (such as TimescaleDB, an extension of PostgreSQL itself, or InfluxDB) typically offer automatic partitioning by time, more efficient compression for time-ordered numeric data, and query functions built specifically for time-series operations (resampling, gap-filling, moving windows).

- **When a relational database is enough:** Daily or hourly bar data for a few thousand instruments comfortably fits in PostgreSQL with the indexing discipline from §12.5; the practical exercise's OHLCV schema is a good example of data that does not need a specialised time-series database.
- **When a time-series database earns its complexity:** Full tick data (potentially billions of rows per day across a large universe) benefits materially from automatic time-partitioning and the compression a purpose-built time-series database provides, keeping both storage cost and query latency manageable at that scale.

12.7 NoSQL (Where Appropriate)

NoSQL databases (document stores, key-value stores, graph databases) relax the rigid, predefined schema a relational database requires, trading some consistency guarantees and query flexibility for schema flexibility and, in some cases, easier horizontal scaling. The name is a historical artefact; the honest framing is not 'SQL vs. NoSQL' but 'which data model fits this specific data.'

Choosing where OHLCV, reference and config data should live

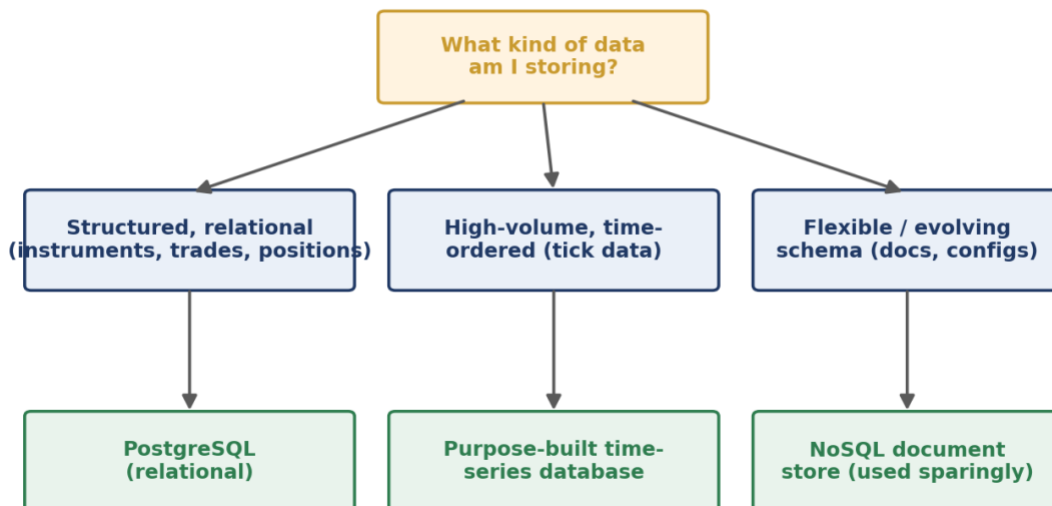


Figure 12.3 — The data itself, not fashion, should determine the storage choice: structured, relationship-heavy data belongs in PostgreSQL; high-volume time-ordered data may justify a time-series database; genuinely flexible, evolving-schema data is the specific case NoSQL is for.

- **A reasonable NoSQL use case:** Storing loosely-structured, frequently-changing configuration documents or unstructured research notes, where a rigid schema would need constant, disruptive migration.
- **A poor NoSQL use case:** Storing OHLCV bars or trade records, where the schema is stable, the data is inherently relational (instruments to bars, orders to fills), and SQL's join and aggregation capabilities are exactly what research queries need — precisely the common mistake in §12.9.

12.8 Worked Exercise — Model Answer

Exercise: Design a PostgreSQL schema for storing multi-asset OHLCV data with proper indexing, and write a query that retrieves a cross-sectional universe efficiently.

The schema (as introduced in Figure 12.1 and Example 12.2)

instruments holds one row per tradeable instrument (slowly changing reference data); ohlcv_bars holds one row per instrument per bar (rapidly growing time-series data), linked by instrument_id, with a composite index on (instrument_id, ts) supporting the most common access pattern.

Model answer — an efficient cross-sectional universe query

```
-- Cross-sectional query: for every instrument in a given asset class,
-- retrieve its most recent closing price as of a specific date --
-- exactly the shape of query a cross-sectional research universe needs.
SELECT i.ticker, b.ts, b.close
FROM instruments i
JOIN LATERAL (
    SELECT ts, close
    FROM ohlcv_bars
    WHERE instrument_id = i.instrument_id
      AND ts <= '2024-06-30'
    ORDER BY ts DESC
    LIMIT 1
) b ON true
WHERE i.asset_class = 'equity'
ORDER BY i.ticker;
```

This query is efficient specifically because of the schema design in §12.3 and the index in §12.5: the LATERAL join's inner query filters on instrument_id and ts together, exactly matching the composite index, so PostgreSQL can find each instrument's latest bar via a fast index lookup rather than scanning the entire ohlcv_bars table once per instrument. Running EXPLAIN ANALYZE on this query (§12.4) should show an index scan, not a sequential scan, on ohlcv_bars — confirming the schema and index are doing their job together, not just in isolation.

12.9 Common Mistakes — Deeper Look

Mistake: Storing time-series data in a schema not indexed by time and instrument, causing slow queries at scale

A table that starts small feels fast regardless of indexing, which is precisely what makes this mistake easy to miss during initial development: a query against ten thousand rows completes quickly whether or not an index exists, and the absence of the composite index from Example 12.4 only becomes visible once the table has grown into the millions or billions of rows Figure 12.2 illustrates — often well after the schema has already been built around, and is expensive to change. The fix is to design the index alongside the schema from the start, based on the query patterns the practical exercise actually needs (per-instrument time-range lookups), rather than adding it reactively once queries have already become unacceptably slow in production.

Mistake: Reaching for a NoSQL solution without a clear reason when a relational database would suffice

OHLCV and trade data is inherently relational — instruments relate to bars, orders relate to fills, portfolios relate to positions — and SQL's join and aggregation capabilities (§12.2) are built precisely for querying exactly these kinds of relationships. Choosing a NoSQL document store for this data because it is newer or more fashionable, rather than because the data genuinely has a flexible or evolving schema (§12.7), typically means re-implementing joins and

aggregations in application code that a relational database would have performed natively, correctly, and far more efficiently.

⚠ Case study: an unindexed tick-data table that quietly stalled research

- A research team stored several years of daily bar data in a single table with no index beyond the default primary key, and queries ran acceptably fast throughout early development against a small, growing dataset.
- As the universe expanded to several thousand instruments and years of history accumulated, a routine cross-sectional query that previously took under a second began taking several minutes, and researchers began quietly avoiding certain analyses rather than waiting on slow queries — exactly the silent capacity cap described in §12.1's opening rationale.
- Adding the composite index from Example 12.4 on the existing table — a single CREATE INDEX statement — reduced the same queries back to sub-second execution, with no change to any application code, illustrating how disproportionate the cost of a missing index can become relative to the cost of adding one.

12.10 Advanced Extensions

Purpose-built time-series databases (e.g. for tick data): beyond the introduction in §12.6, production tick-data systems typically rely on automatic time-based partitioning (so old data can be compressed or archived without touching recent, actively-queried partitions) and continuous aggregates (pre-computed rollups, such as 1-minute bars derived automatically from raw ticks, refreshed incrementally as new data arrives).

Column-oriented storage suits analytical, single-column queries

Row-oriented: reads a whole row even for one column							Column-oriented: reads only the 'close' column needed						
ts	instr	open	high	low	close	vol	ts	instr	open	high	low	close	vol
9:30	AAPL	189.1	189.4	188.9	189.2	1.2M	9:30	AAPL	189.1	189.4	188.9	189.2	1.2M
9:30	MSFT	402.5	402.9	402.1	402.7	0.8M	9:30	MSFT	402.5	402.9	402.1	402.7	0.8M
9:30	GOOG	141.0	141.3	140.8	141.1	0.6M	9:30	GOOG	141.0	141.3	140.8	141.1	0.6M

Figure 12.4 — Row-oriented storage (used by PostgreSQL by default) must read an entire row even to retrieve a single column; column-oriented storage, common in analytical and time-series-oriented systems, reads only the specific column a query needs — a large efficiency gain for research queries that scan one metric (e.g. close price) across millions of rows.

- **Columnar storage formats:** Formats such as Parquet store each column contiguously on disk rather than each row, which dramatically speeds up analytical queries (aggregating one column across many rows) at some cost to the speed of retrieving a single, complete row — the opposite trade-off profile from PostgreSQL's default row-oriented storage, and complementary to it for large-scale offline research workloads.

12.11 Professional Applications

- **Historical data storage:** The schema design and indexing discipline in §12.3–§12.5 is what keeps multi-year, multi-asset backtests (§6.1–§6.7) computationally feasible as a firm's historical data grows.
- **Feature stores:** A structured, well-indexed store of precomputed research features (signals, risk metrics) that both research and production systems can query consistently, avoiding the drift that arises when research and production independently recompute the same feature slightly differently.

- **Production reference data:** Low-latency, always-available access to instrument reference data (tickers, contract specifications, corporate actions) and current positions is a hard requirement for any real-time trading system, directly connecting this section's schema and indexing discipline to the low-latency needs described in §12.1.